



上海科技大学
ShanghaiTech University

From Chaofan Li



立志成才报国 格天



上海科技大学
ShanghaiTech University

Practising

纸上得来终觉浅，绝知此事要躬行。

- Today's recitation will include several exercises. We welcome all of your participation.



立志成才 报国强民



- Lecture notes
 - undefined behavior, integer promotion, binary number representation
- Arithmetic Types
 - `sizeof`, floating-point error
- Literals
 - integer/floating-point/character literals
- Operators
 - precedence, associativity, order of evaluation
 - bitwise operators





- Undefined behavior ->
- *Integer promotion ->
- *Binary number representation: sign-and-magnitude, one's complement, two's complement

Note: The items with asterisk(*) are beyond the scope of this class.





Undefined Behavior ->

There are no restrictions on the behavior of the program.

- memory accesses outside of array bounds (covered later)
- **signed integer overflow**
- null pointer dereference (covered later)

Note: unspecified behavior

- Two or more behaviors are permitted and the implementation is not required to document the effects of each behavior





Integer Promotion ->

- Definition: Integer promotion is the implicit conversion of a value of any integer type with rank less to the value of type `int` or `unsigned int`.
- Application:
 - as part of usual arithmetic conversions
 - Ex. promotion of operands of relational operator `==`

```
#include <stdio.h>

int main(void){
    char a = 251;
    unsigned char b = a;

    if (a == b)
        printf("Same.\n");
    else
        printf("Not Same.\n");

    return 0;
}
```



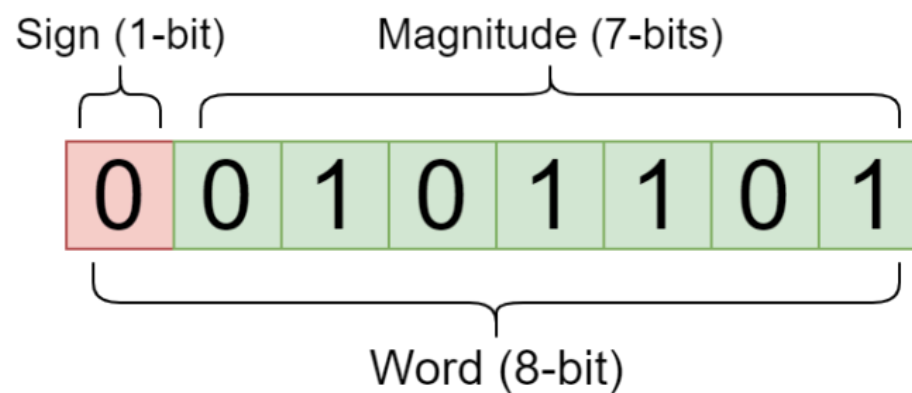
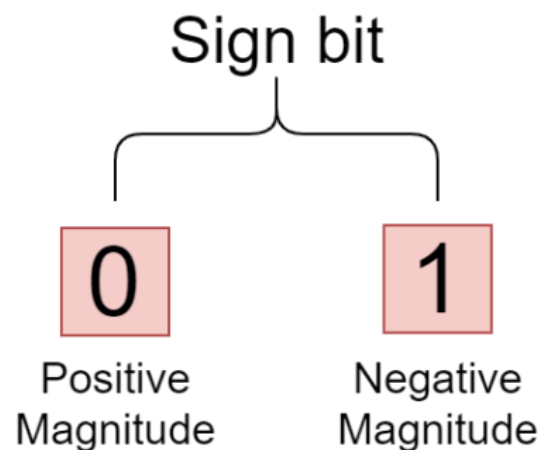


上海科技大学
ShanghaiTech University

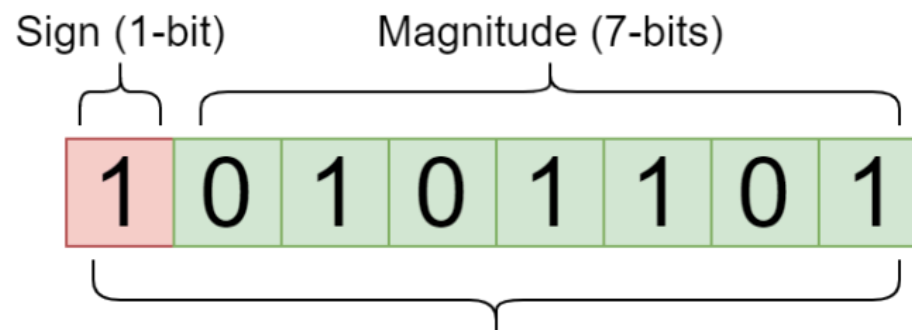
- Sign-and-Magnitude
- One's complement
- Two's complement



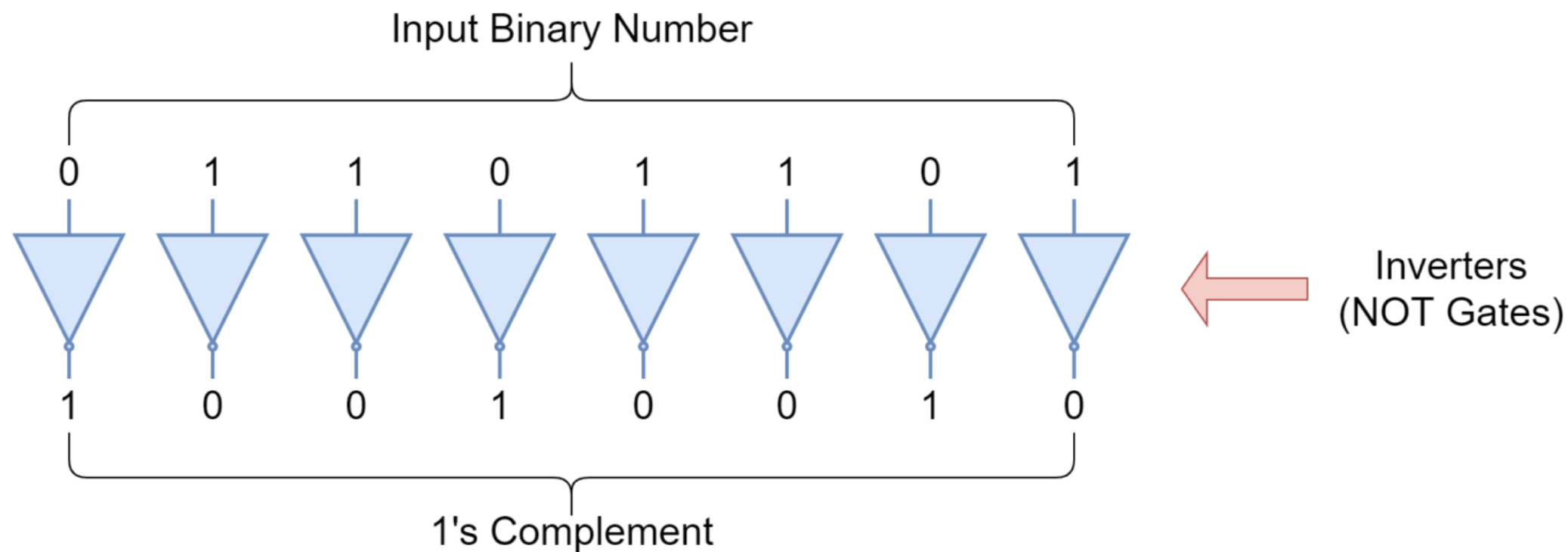
立志成才 报国强民



= 45



= - 45





Sign bit

Magnitude

1	1	1	1	1	1	0	0	-4
---	---	---	---	---	---	---	---	----

0	0	0	0	0	1	0	0	+4
---	---	---	---	---	---	---	---	----



Arithmetic Types ->

- Overview
- Width of integer types and `sizeof`
- Error of floating-points





Note: Sometimes, **characters** and **Boolean values** are also considered as **integers**.

signed
char

unsigned
char

char

bool

(signed)
short (int)

unsigned
short (int)

char8_t

float

signed / int /
signed int

unsigned (int)

char16_t

double

(signed) long (int)

unsigned long (int)

char32_t

long double

wchar_t

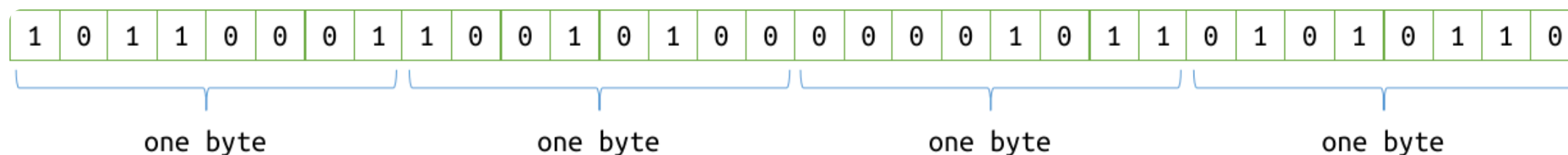
(signed) long long (int)

unsigned long long (int)



sizeof

A 32-bit number: $2979269462_{\text{ten}} = 10110001100101000000101101010110_{\text{two}}$.



->

- A byte is 8 bits on most platforms.
- `sizeof(T)` is the number of **bytes** that `T` holds.
 - `sizeof()` returns a value of type `size_t` ->. (covered later)
- The conversion specifier for `size_t` is `%zu`:

```
printf("%zu\n", sizeof(double));
```



sizeof

type	width (at least)	width (usually)
short	16 bits	16 bits
int	16 bits	32 bits
long	32 bits	32 or 64 bits
long long	64 bits	64 bits

- `sizeof(signed T) == sizeof(unsigned T)`
- **It is also guaranteed that** `sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)`.
- You can print the width of each type to see how your machine implements these types.





上海科技大学
ShanghaiTech University



立志成才报国 裕民



Define a function `midpoint` that takes two 64-bit integers and returns their average. Be careful when choosing **the return type and parameters' types**.

- After defining the function, calculate the average of `9007199254740993` and `9007199254740993` using your function.





上海科技大学
ShanghaiTech University

inevitable (必然

的)

- When checking if a floating-point number is zero, you must use $|x| < \epsilon$, where ϵ could be a small number like 10^{-8} .
- To check if two floating-point numbers are equal, use $|a - b| < \epsilon$.



立志成才 报国 裕民



- Integer Literals (整型字面值)
- Floating-point Literals (浮点数字面值)
- Character Literals (字符字面值)
- ~~String Literals (字符串字面值)~~
- ~~Compound Literals (复合字面值)~~





- Every expression in C has a specific type determined at compile time.
 - An expression is a sequence of operators and their operands, that specifies a computation.
- For example, `int a = 42; double b = 3.14`
 - What are the types of `a` and `b`?
 - What are the types of `42` and `3.14`?





Writing values explicitly like `42` are known as literals.

- Literals are immutable. (unchangeable)
- A literal is simply a value. (字面值-字面即是值；把值写在脸上的就是字面值)





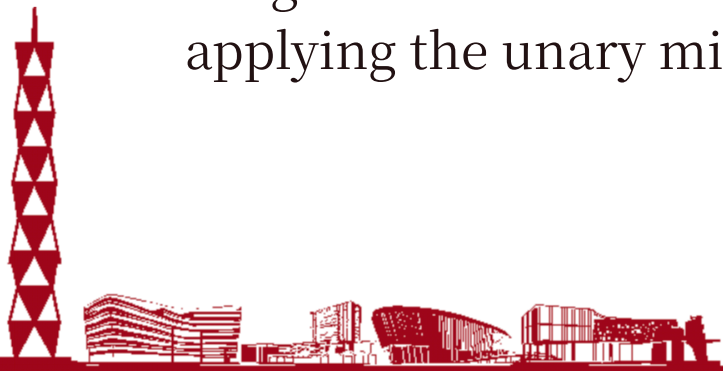
42

100L

0LL

405uL

- Suffix: `L`(long), `LL`(long long), `uL`(unsigned long) — these indicate the type of the literal and are case-insensitive. (大小写不敏感)
 - It is recommended to avoid mixing `l` and `L`, such as in `LL` or `LL`; instead, consistently use `L`.
 - Suffixes can be combined freely.
- If there's no suffix in an integer literal, the default type is `int`.
 - If the value exceeds the range of `int`, it will incrementally match to `int`->`long`->`long long`->`__int128`. If `__int128` is not supported, an error will occur (ill-formed).
- "Negative literals" do **not exist**; for example, `-42` represents an expression formed by applying the unary minus operator `-` to the literal `42`.





- `0xBAADF00D`, `052`, `0b101010` (C23),
- **Prefixes** (case-insensitive):
 - Hexadecimal literal: `0x`
 - Octal literal: `0`
 - Binary literal: `0b` (since C23)
 - The C++ compiler has supported binary literal since C++14.





- 3.14, 3.14f, 3.14l, 1e8, 3e-8
- Suffixes (**case-insensitive**):
 - default: double
 - f : float
 - l : long double
- **Scientific notation is always a floating-point number.**
 - For instance, 1e8 represents 10^8 , but it is of type double, not int.





'a' '\n' '1'

- The type is `int`.
 - In C++, it is represented as `char`.

“ Character literals are individual characters of type `int` suitable for conversion to a character type or of type `char8_t`, (since C23) `char16_t`, `char32_t`, or (since C11) `wchar_t`.

- Example: `char c = 'a';`
 - The content stored in `c` is the integer 97, which is the ASCII -> code for 'a', not an image.
 - **There's no "conversion" between a character and its corresponding ASCII code! They are essentially the same.**





Given that the character 'c' is a digit ('1', '2', ...), determine the numerical value(int) it represents.

- Input: c - A single character representing a digit.
 - If c is not a digit, the behavior is undefined.
- Output: The corresponding numerical value as an **integer**.





- If you can only use `getchar()` to read a **character**, how would you read a non-negative **integer**?
 - `getchar()` : CppReference ->
- Enhance the robustness: What if at the beginning of the input, there's some noise like leading whitespaces?
 - using functions declared in the standard library header `<ctype.h>`
 - character classification functions ->





上海科技大学
ShanghaiTech University

- Precedence, Associativity, Order of Evaluation
- Bitwise operators



立志成才报国 裕民



- Precedence: determines which operation to **perform first**
 - The expression $f() + g() * h()$ is interpreted as $f() + (g() * h())$, not as $(f() + g()) * h()$.
- Associativity: how **operators of the same precedence** are grouped in an expression
 - The expression $f() - g() + h()$ is interpreted as $(f() - g()) + h()$, not as $f() - (g() + h())$.
- Precedence and Associativity Tables - CppReference ->
- a classic question: Piazza ->





Associativity: how **operators of the same precedence** are grouped in an expression

- Left Associativity: operations are grouped from the left
 - Operators with the same precedence are associated from left to right; the operator that appears first binds first.
 - For example, the expression $f() - g() + h()$ is interpreted as $(f() - g()) + h()$, not as $f() - (g() + h())$.
- Right Associativity: operations are grouped from the left
 - For example, the assignment operator $=$: $a = b = c$
 - Step 1: $a = (b = c)$
 - Step 2: $b = c$ returns b (equal to c)
 - Step 3: $a = b$



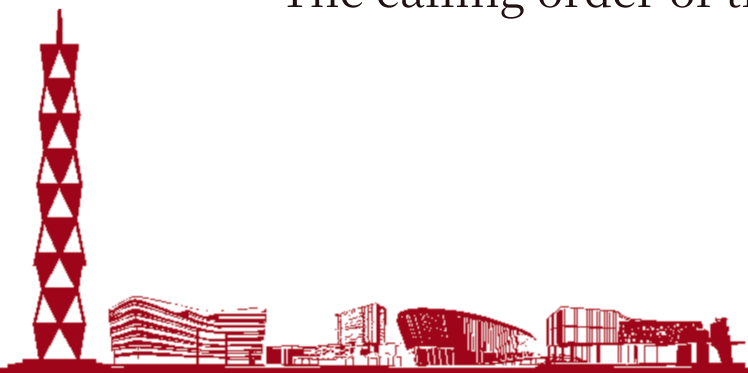


- Very important: Order of Evaluation is independent from both precedence and associativity.
- Very Very important: Order of evaluation of the operands of any C operator is unspecified (except where mentioned later).





- What is the calling order for `f()`, `g()`, and `h()` in the previous examples?
(`f() - g() + h()`, `f() - (g() + h())`, ...)
 - It is **unspecified** (未指定) .
 - You most likely cannot define precisely what "unspecified" means; perhaps some students have already noted down "this is unspecified" in their notebooks.
 - I hope you can understand what truly happens during compilation; the answer is right at your fingertips order of evaluation-CppReference ->.
- Another example: `func(f(), g(), h())`
 - The calling order of the functions `f()`, `g()`, and `h()` here is also unspecified.





If the order of evaluation of two expressions **A** and **B** is unspecified, and they either:

- Both modify the value of a variable, or
- One modifies the value of a variable and the other reads that value,

then this constitutes **undefined behavior**.

- Example: What is the result of `i = i++ + 2`?
 - Once you start analyzing whether it is `+1` or `+2`, you have fallen into the **trap of undefined behavior**.
(just like to analyze `double pi = 3.14; printf("%d\n", pi);`)
 - `++` modifies `i`, and the assignment also modifies `i`. The order of these modifications is unspecified, hence it is **undefined behavior**.

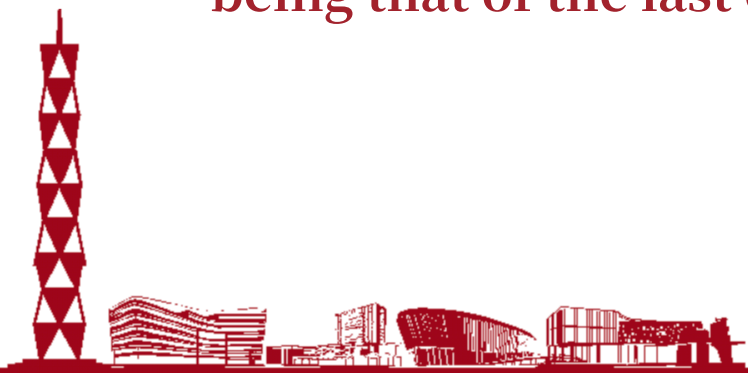




Operators with determined evaluation order

Among common operators, only four have a determined evaluation order: `&&`, `||`, `?:`, and `,`.

- `&&` & `||`: short-circuit evaluation(or minimal evaluation, McCarthy evaluation)
 - evaluate the left side first; **the right side is not evaluated if not necessary**
- `cond ? t : f`
 - evaluate `cond` first, then evaluate `t` or `f` based on the truth value of `cond`
 - only one of `t` or `f` is evaluated
- `,`: Executes multiple expressions in sequence, with **the whole comma expression's value being that of the last expression in the series.**





```
int main(void){  
    int a = 1;  
    int b = (a++, a + 100, a - 10);  
}
```

What is the value of **b** in the program above?





~, &, ^, |, <<, >>, and compound assignment operators (复合赋值运算符) &=, |=, ^=, <<=, >>=

Operator	Operator Name	Example	Result
~	Bitwise NOT	~a	the bitwise NOT of a
&	Bitwise AND	a & b	the bitwise AND of a and b
	Bitwise OR	a b	the bitwise OR of a and b
^	Bitwise XOR	a ^ b	the bitwise XOR of a and b
<<	Bitwise Left Shift	a << i	a left-shifted by i
>>	Bitwise Right Shift	a >> i	a right-shifted by i

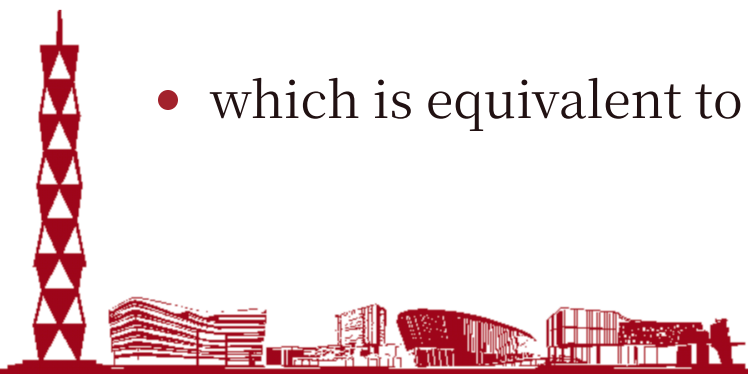


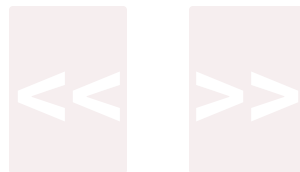


- $\sim a$: Returns the result of flipping every binary bit of a .
 - For example, the bitwise NOT of 10010110_{two} equals 1101001_{two} .
- The i^{th} bit of $a \& b$ is 1 if and only if **both the i^{th} bits of a and b are 1**.
- The i^{th} bit of $a | b$ is 1 if and only if **at least one** of the i^{th} bits of a and b is 1.
- The i^{th} bit of $a \wedge b$ is 1 if and only if the i^{th} bits of a and b are **different**.

Example: $a \& 1$:

- a bitwise AND between a and $\underbrace{0 \cdots 00}_{n-1 \text{ bits}} 1_{\text{two}}$,
- which is equivalent to $a \% 2$.





Assume `a` is an unsigned integer.

- `a << i` returns the result of **shifting the binary bits of `a` to the left by `i` positions**.
 - For example, `a << 1` is equivalent to `a * 2`, and `a << i` is equivalent to multiplying `a` by 2^i .
 - **Bits that overflow on the left are discarded.**
- `a >> i` returns the result of **shifting the binary bits of `a` to the right by `i` positions**.
 - For example, `a >> 1` is equivalent to `a / 2`.
 - **Bits that overflow on the right are discarded.**





上海科技大学
ShanghaiTech University



立志成才报国 裕民



`1 << n`

- But `1` is an `int` literal.
- If `n` is large, you might need `1u << n`, `1ll << n`, etc.





How do you obtain the i -th bit of an unsigned integer x ?

- Let's agree that i is in the range $[0, N)$, where N is the total number of bits in x 's type. The 0-th bit is the rightmost bit (LSB, least significant bit).

```
int test_bit(unsigned x, unsigned i) {  
    // TODO  
}
```





How do you flip the i -th bit of an unsigned integer x ?

```
unsigned bit_flip(unsigned x, unsigned i) {  
    // TODO  
}
```





How do you extract the bit range $[low, high)$ from an unsigned integer x ?

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    // TODO  
}
```





Given n non-negative 32-bit integers, among which **one number appears an odd number of times, while others appear an even number of times.**

- Find the number that appears an odd number of times.





Answer: $x_1 \oplus x_2 \oplus \cdots \oplus x_n$.

```
int main(void){  
    int n;  
    scanf("%d", &n);  
    unsigned result = 0;  
    while (n--) {  
        unsigned x; scanf("%u", &x);  
        result ^= x; // How?  
    }  
    printf("%u\n", result);  
    return 0;  
}
```



“ Given n non-negative 32-bit integers, among which **one number appears an odd number of times, while others appear an even number of times**. Find the number that appears an odd number of times.

- First, prove that XOR has commutative and associative properties:
(you may prove by truth tables; learned in discrete mathematics, digital circuit)
 - $a \oplus b = b \oplus a$
 - $(a \oplus b) \oplus c = a \oplus (b \oplus c)$
- Moreover, $a \oplus a = 0$, $a \oplus 0 = a$.
- Thus, for $x_1 \oplus \cdots \oplus x_n$, by arbitrarily rearranging the terms to combine identical numbers, they cancel out, leaving the number that appears an odd number of times.





Magic of XOR - Answer

“ Given n non-negative 32-bit integers, among which **one number appears an odd number of times, while others appear an even number of times**. Find the number that appears an odd number of times.

Another interpretation of the answer: Consider the number r that appears an odd number of times.

- If the i th bit of r is 1, it implies there are an odd number of 1s in the i th bit of all the numbers.
- If the i th bit of r is 0, it implies there are an even number of 1s in the i th bit of all the numbers.

Thus, we need an operation that turns an even number of 1s into 0 and an odd number of 1s into 1. Essentially, this is addition without carrying in binary context.

• The addition is XOR.



上海科技大学
ShanghaiTech University

From Chaofan Li



立志成才报国 裕民